APPLIED ARTIFICIAL INTELLIGENCE

CaseTrace

Evidence-Grounded Document Investigation

An Investigator–Verifier RAG Platform
with Source Attribution and Human Review



MAROC Abdelhakim Fouad

Technical project report
August 2026

System
LangGraph, FastAPI, FAISS

Evaluation
Synthetic dossiers and recorded LLM runs

Application
Document consistency analysis

Abstract

Document review involves more than generating a fluent summary. An analyst must connect statements to identifiable sources, distinguish missing information from contradictions, check calculations, and preserve the reasoning behind follow-up requests. I developed CaseTrace to support this process through a local web platform that combines retrieval-augmented generation (RAG), a bounded investigator–verifier workflow, deterministic citation checks and human review.

The investigator can retrieve passages, read a source page and perform restricted numerical operations before producing a structured report. A second language-model role challenges the findings against the cited pages. The application records actions, evidence, verification outcomes and model usage, and exports reports in PDF, Markdown and JSON. I also implemented a single-pass RAG baseline and a deterministic offline mode to separate the effects of orchestration from basic document processing.

The evaluation uses three synthetic dossiers containing nine PDFs. On eight source-location queries, the expected document ranks first in the saved retrieval results. On the consistent dossier, both live approaches produce three source-supported confirmations and no issue categories. The agent workflow uses three calls and 6,289 tokens, compared with one call and 1,906 tokens for the baseline. A successful earlier agent run on the contradictory dossier demonstrates model-selected arithmetic and a separate verifier assessment. Later quota failures are retained as execution outcomes rather than replaced by successful historical results.

These observations establish the feasibility and inspectability of the implemented workflow. They do not demonstrate an accuracy advantage over single-pass RAG: the corpus is small, was used during development, and fits entirely within the initial retrieval window. The principal outcome is an executable platform with explicit evidence handling and a reproducible foundation for a larger evaluation.

Keywords: agentic RAG; document intelligence; multi-agent workflows; provenance; citation validation; human-in-the-loop review; LLM evaluation.

Report scope

This report describes the implemented system and the saved experimental record. The quantitative comparison uses Gemini 2.5 Flash runs recorded on 9 October 2026. A subsequent configuration change to Gemini 3.8 Flash was checked with a minimal successful request; the benchmark was not rerun with that model. I keep these two kinds of evidence separate throughout the report.

Contents

1 Introduction and objectives 1

2 System architecture 2

3 Data model and document processing 3

4 Retrieval and evidence grounding 4

5 Investigator–verifier orchestration 5

6 Platform, reliability and human review 6

7 Experimental design 7

8 Evaluation measures and coverage 8

9 Results and resource trade-offs 9

10 Case study: the Riverside dossier 10

11 Software validation and reproducibility 11

12 Discussion and future work 12

13 Conclusion 12

A Execution details and API reference 13

B Evidence manifest 14

1 Introduction and objectives

1.1 Motivation

A dossier can contain documents that are individually plausible but mutually inconsistent. A declaration may give one incident date while an inspection report gives another. A repair estimate may contain a correct subtotal and an unexplained final amount. A repeated invoice number may indicate a resubmitted attachment rather than a second payment request. Identifying these distinctions requires a combination of retrieval, comparison, arithmetic and contextual interpretation.

I chose this setting because it exposes an important weakness of unstructured language-model answers: a convincing explanation can be difficult to audit. My objective was to make each finding inspectable, with a document identifier, a page number, an exact quotation and a concrete follow-up action. The analyst remains responsible for assessing the report and deciding what evidence to request.

1.2 Project questions

I organized the project around three questions:

1. Can a language-model investigator use a small, restricted tool set to examine a dossier and produce a structured, source-linked report?
2. Can a verifier expose unsupported interpretations and trigger a bounded revision while preserving an audit record?
3. What additional calls, tokens and latency does the investigator–verifier workflow require compared with single-pass RAG on the same documents?

The first two questions concern system behavior; the third concerns an empirical trade-off. A verifier is not assumed to improve accuracy merely because it adds a second model call.

1.3 Implemented contributions

The project integrates document ingestion, page-preserving retrieval, model-driven tool selection, structured generation, deterministic source checks and human review in one application. The evaluation infrastructure records both successful and unsuccessful executions, including document hashes and model usage. An offline mode supports demonstrations without a model connection, while remaining explicitly distinguishable from the live agent workflow.

The technical stack is Python, FastAPI, LangGraph, FAISS, Pydantic and SQLite, with a browser interface written in HTML, CSS and JavaScript. Gemini is the provider used for the recorded live experiments. A compatible local endpoint and Gemini semantic embeddings are configurable extensions, but their availability is not treated as evidence that they were evaluated.

1.4 Relation to existing approaches

RAG combines language generation with retrieved external information [1]. CaseTrace uses the same general retrieve-then-generate principle, but does not train a retrieval model or reproduce the original paper's learning procedure. Its default retriever is a transparent lexical representation.

The investigator's action loop is related to the broader idea of combining language-model decisions with external actions [2]. Here, I record structured actions and tool outputs; I do not request or store private chain-of-thought. LangGraph implements the control flow, and FAISS provides exact vector similarity search [4, 3].

Design principle

A report should make it easy to check where a statement came from, what operation was performed, and which questions remain unresolved.

2 System architecture

2.1 Separation of responsibilities

The browser manages dossier selection, uploads, investigation requests and report inspection. FastAPI validates requests and exposes a small set of endpoints. A single background worker runs the investigation while the interface polls for progress. SQLite persists the case and run records; original files remain in local storage under generated identifiers.

The retrieval component and calculator provide evidence and deterministic operations. The model adapter handles provider requests, JSON decoding, usage accounting and failures. LangGraph coordinates the investigator, verifier and optional revision. This separation allows API and workflow tests to replace live model calls with controlled responses.

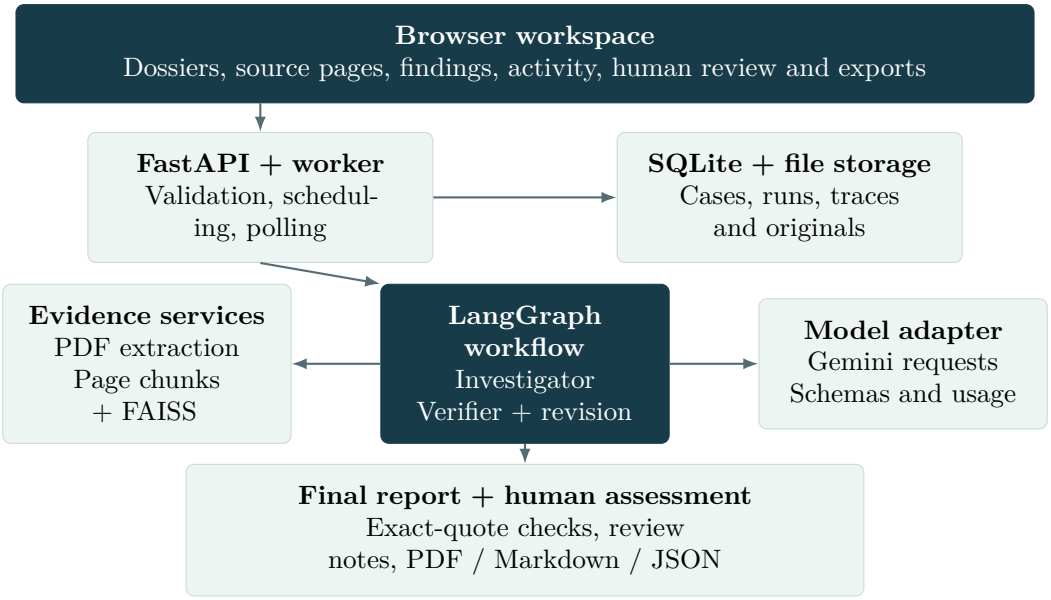


Figure 1: Implemented architecture. Documents and run records are stored locally. Model calls transmit the selected evidence to the configured provider after user confirmation.

2.2 Execution lifecycle

A run is initially queued, then marked running. Events are appended as retrieval, model requests, tool execution and verification proceed. Completion stores a report and usage summary; a provider or validation error stores a failure instead. Cancellation is checked between steps. It cannot retract a request already being processed by a remote provider.

On application restart, unfinished queued or running records are marked interrupted. This avoids presenting an incomplete execution as a completed investigation. The single-worker design bounds concurrent model use but is intended for a local, single-user workload rather than a distributed service.

3 Data model and document processing

3.1 Core entities

Entity	Principal fields	Purpose
Case	Identifier, title, creation time, document list	Defines the evidence boundary of an investigation.
Document	Identifier, filename, SHA-256, extracted pages, warnings	Connects source bytes to searchable page text.
Run	Objective, mode, configuration, status, events, result	Preserves the execution and its outcome.
Finding	Category, severity, explanation, citations, next step	Expresses an actionable observation.
Citation	Document identifier, page number, quotation	Locates textual evidence.
Review	Finding index, verdict and justification	Records the verifier's assessment.
Human review	Status, note and timestamp	Records the user's assessment of the report.

Table 1: Main application entities and their responsibilities.

Cases and runs are serialized into SQLite records using parameterized statements. Original document bytes are saved using generated identifiers rather than user-supplied paths. Each run also retains the source identifiers and hashes that were used for its analysis. The JSON export therefore contains more provenance information than the presentation-oriented PDF export.

3.2 Extraction and input limits

I use `pypdf` for text-based PDFs. UTF-8 TXT, Markdown, CSV and email text are also accepted, each represented as a single text page. Encrypted PDFs, unusable text, unsupported extensions and oversized files are rejected. Some PDFs may contain both readable and empty pages; extraction warnings are retained in the document record.

The application accepts up to 12 documents per case, 5 MiB per file and 40 pages per PDF. Extracted text is limited to 200,000 characters per document and 500,000 characters per case. These limits bound ordinary workloads; they are not a substitute for a hardened upload service. Scanned images require an OCR stage, which is outside the implemented extraction pipeline.

3.3 Finding representation

The report uses five finding categories: contradiction, arithmetic, duplicate, supported and missing information. The first three describe an inconsistency or repeated reference; `supported` expresses a positive confirmation. Missing-information findings identify an absent item without assuming that every unprovided document was required.

Pydantic validates the object structure, supported categories, page numbers and size limits. A report contains at most 12 findings; each finding contains at most six citations. These constraints limit the output presented to the user and make downstream rendering predictable.

Interpretation of a duplicate

A repeated invoice reference is a document-level observation. Its meaning depends on context: a copy explicitly labelled as a resubmission should not be described as evidence of a second payment request.

4 Retrieval and evidence grounding

4.1 Page-preserving chunking

For each extracted page, the retriever creates chunks of up to 1,100 characters at a stride of 850 characters. Adjacent chunks can therefore overlap by 250 characters. Every chunk retains its document identifier, source filename and page number. Chunks never cross a page boundary.

This design makes source attribution straightforward, but character-based cuts may split words, tables or sentences. The current corpus is short enough that each PDF produces a single chunk. Larger and more structurally complex documents would require an evaluation of sentence-aware or layout-aware segmentation.

4.2 Local lexical representation

The default representation is lexical, not a learned embedding. Let N denote the number of chunks in one case and $\mathrm{df}(t)$ the number containing token t. I compute

$$\mathrm{idf}(t) = \log\left(\frac{1+N}{1+\mathrm{df}(t)}\right) + 1. \tag{1}$$

Word tokens are mapped to 2,048 buckets with a deterministic hash. For chunk c, the unnormalized vector is

$$x_{c,j} = \sum_{t \in c:\, h(t)=j} \mathrm{tf}(t,c)\,\mathrm{idf}(t), \tag{2}$$

where the sum is over distinct terms and tf is their occurrence count. I then normalize $\hat{x}_c = x_c / \|x_c\|_2$ when the norm is nonzero. Queries use the same case-specific vocabulary statistics, with a default weight of one for unseen tokens.

FAISS `IndexFlatIP` performs exact inner-product search on the normalized vectors [3]. Its score is consequently a cosine similarity for nonzero vectors. Hash collisions and sensitivity to vocabulary are known limitations: a semantically relevant paraphrase can be missed if it shares few terms with the source.

4.3 Retrieval within the workflow

The initial stage retrieves up to six chunks for the objective. An investigator search action retrieves up to four additional chunks. The investigator can also request an identified page directly, with a 12,000-character response bound. Evidence retained in graph state is limited to the last 20 entries.

An optional semantic path requests 768-dimensional Gemini embeddings with separate retrieval-document and retrieval-query task types. It requires external transmission of chunks and queries, and adds requests beyond the chat-model counter. This path was implemented but was not exercised in the benchmark reported here. All reported retrieval results use the local lexical representation.

4.4 Deterministic citation validation

For each quotation, the checker resolves the referenced document and page, normalizes whitespace and letter case, and tests substring membership. It flags missing pages, unknown documents and quotations not present in the source. Non-missing-information findings must contain a citation; the schema allows an uncited missing-information observation, so this exception must not be interpreted as source verification.

The check establishes textual presence, not entailment. A quote can be genuine while its interpretation is incorrect. This distinction motivates the verifier and the source viewer, rather than treating a successful string match as a factuality guarantee.

5 Investigator–verifier orchestration

5.1 Investigator tools

The investigator receives the objective, a document inventory, retrieved passages and prior tool results. It chooses a structured action through an application-level JSON protocol. The allowed actions are listed in Table 2; no generated code, arbitrary URL or shell command is executed.

Action	Input	Behavior
`search`	A text query	Retrieves up to four chunks from the current case.
`read_page`	Document identifier and page	Returns bounded text from a known page.
`calculate`	Operation and two arguments	Uses Decimal arithmetic or an ISO-date difference.
`finish`	A report object	Submits the structured draft for verification.

Table 2: Restricted investigator action space.

The calculator supports addition, subtraction, multiplication and the number of days between two ISO dates. Decimal arithmetic avoids binary floating-point rounding in the money-like values used here. The model still has to select the relevant values and interpret the result correctly.

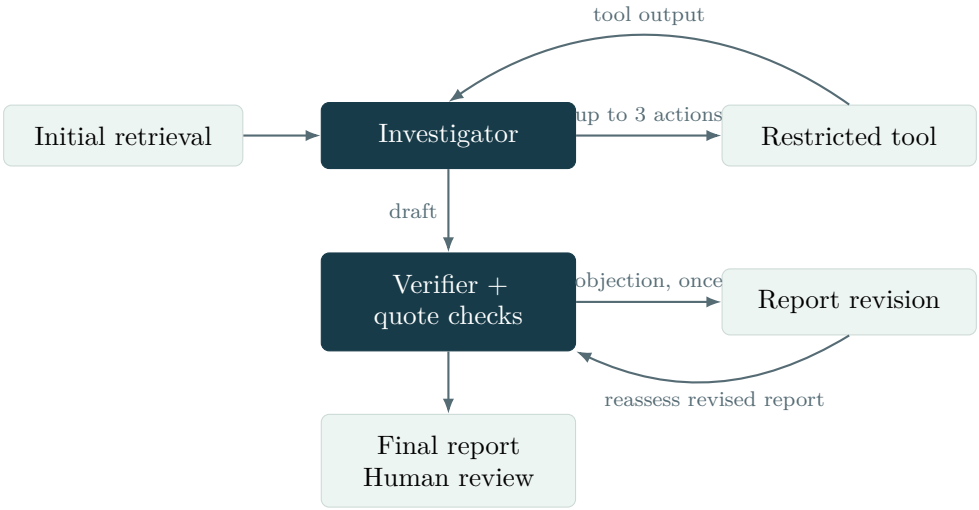


Figure 2: Bounded graph execution. The verifier may request one semantic revision. Remaining objections are retained in the final result. A global call budget also constrains execution.

5.2 Verifier and stopping conditions

The verifier receives the draft, exact-quote checks, initial retrieval and source pages fetched independently from the draft's citation identifiers. It must assess every finding index exactly once using `supported`, `revise` or `unsupported`, with a justification. This is a separate role and evidence-reading step, not an independently trained model: both roles use the same configured provider and can share errors.

After three investigator tool actions, the next step requests a final report. A malformed investigator report can trigger one constrained schema repair. An unsupported finding or invalid quotation can trigger one semantic revision followed by a second verifier pass. The default global limit is eight model calls; reaching it produces a recorded failure rather than an unbounded loop.

6 Platform, reliability and human review

6.1 User workflow

The analyst creates a dossier from local files or loads the synthetic examples, enters an objective and selects an execution mode. Live execution requires confirmation that excerpts may be transmitted to the configured model. The browser polls the run record to show progress while the backend continues processing.

The interface separates findings, agent activity and evaluation. Each finding exposes its evidence and next step. A quotation opens the corresponding page text in a source panel. The activity view records short action descriptions, arguments, results, structured model responses and verifier assessments. The history lists past runs so that an error does not erase an earlier analysis.

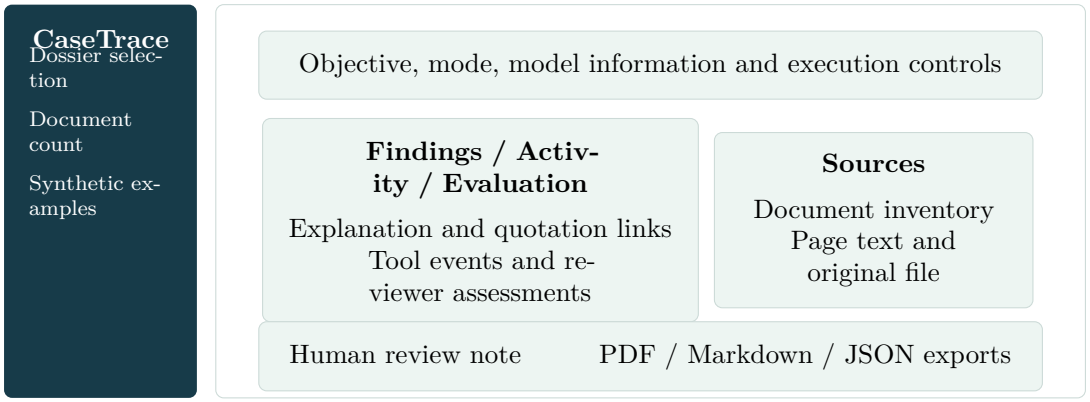


Figure 3: Schematic organization of the implemented interface. This diagram represents the interaction layout rather than a browser screenshot.

6.2 Structured output and error handling

Reports and verifier outputs use a JSON-schema response format with local Pydantic validation. Deep string and array bounds exceeded the provider's constrained-decoding budget during development, so I simplify these bounds in the serving schema while keeping the complete validation locally. Investigator actions use JSON-object mode and are checked against the action allowlist.

The adapter records attempted calls and provider-reported input, output and total tokens. Optional embedding requests are not included in these chat counters. Provider errors, invalid JSON, schema failures and exhausted call budgets remain visible as failures. HTTP 429 terminates the run without an automatic provider switch; the application does not change account billing settings.

6.3 Human assessment and local boundaries

A completed report can be marked unreviewed, accepted, rejected or requiring follow-up, with a saved note. Acceptance concerns the report, not an insurance eligibility or payment decision. Exports preserve either the presentation of findings or the fuller execution record.

The application is bound to localhost by its launcher and rejects unexpected hostnames and cross-origin writes. The model key remains on the server and is excluded from exported configuration. There is no authentication or multi-user authorization layer. These choices suit a local prototype; wider deployment would require a separate access-control and data-protection design.

7 Experimental design

7.1 Synthetic corpus

I created three fictional dossiers with nine one-page PDFs. Their expected labels are kept in separate JSON files that are not supplied to the model. Table 3 summarizes the corpus.

Dossier	PDFs	Construction and expected observations
Harbor workshop	3	Agreement on incident date; labour and materials sum to the stated total; required estimate present. No issue category expected.
Riverside studio	4	Incident dates differ by two days; final total exceeds line items by EUR 200; invoice copy explicitly describes a benign resubmission.
Hillcrest office	2	Declaration and incident report agree; the explicitly requested repair estimate is absent.

Table 3: Evaluation dossiers. All entities, incidents and amounts are fictional.

The dataset is intentionally controlled: each discrepancy has an identifiable source and can be checked manually. It was also used during implementation and debugging, so it is not a held-out test set. The narrow document style favors lexical matching and the offline field rules.

7.2 Compared modes

Single-pass RAG retrieves the initial evidence and requests one structured report. It performs deterministic citation checks but has no tool loop or second-agent review.

Investigator–verifier uses the graph described above, starting with the same objective and initial retrieval. Its extra operations can improve inspectability, but may also increase cost or introduce new errors.

Offline template checks parse explicit field–value lines and compare dates, totals, invoice references and required document types. This mode has no model calls and does not interpret arbitrary objectives. It validates the ingestion/report path and provides an offline demonstration, rather than serving as a general-purpose language-understanding baseline.

7.3 Configuration and recorded measurements

The saved benchmark uses `gemini-2.5-flash`, temperature zero, local lexical vectors, at most 3,500 output tokens per request and an eight-call budget. Provider reasoning is disabled for that model in the adapter. The current record contains one retained attempt per executed dossier/mode pair; older available development attempts are stored separately.

The measurements are source retrieval, predicted issue categories, quote-check outcomes, call count, token usage, elapsed time and execution status. I retain unsuccessful attempts because operational availability is part of the behavior of a provider-dependent system. The later migration to `gemini-3.8-flash` addressed a new-project access restriction and passed a minimal JSON request. It does not change the model identity of the benchmark runs presented here.

8 Evaluation measures and coverage

8.1 Source retrieval

Each of eight queries has one expected source document. I define document-level recall at rank k as

$$R@k = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}\{d_q^\star \in \mathrm{TopDocs}_k(q)\}. \tag{3}$$

The saved ranked filenames allow both $R@3$ and $R@1$ to be computed. $R@3$ is the original evaluator output; $R@1$ is derived from the same saved rankings for this report, without issuing new retrieval or model requests.

8.2 Category agreement and quotation presence

For each dossier, let E be the expected set of issue categories and P the predicted set after excluding `supported`. I record the counts $|E \cap P|$, $|P \setminus E|$ and $|E \setminus P|$: matched, extra and missed categories. This avoids assigning an artificial precision or recall of one to a clean dossier with empty sets.

Category agreement is coarse. It does not distinguish multiple errors of the same kind, and a benign copy might reasonably be expressed as either a duplicate observation or a supported clarification. A separate quote-valid finding count records whether all listed quotations pass the deterministic checker. Neither metric measures complete semantic correctness.

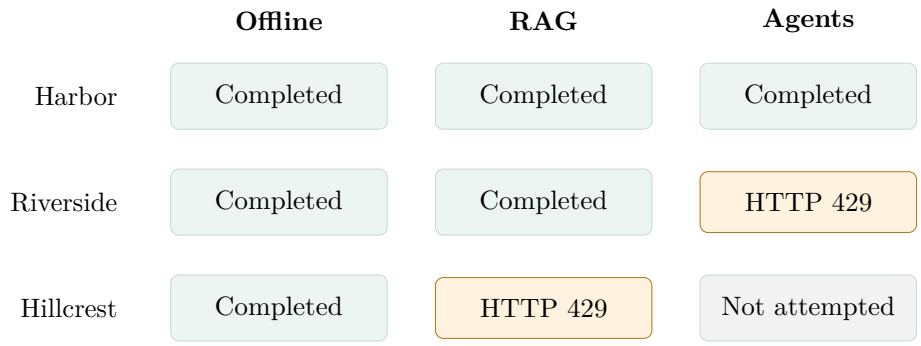


Figure 4: Coverage of the current evaluation record. An earlier successful Riverside agent run exists in the development history; it is analyzed separately and does not replace the failed current attempt.

8.3 Availability and the limits of the retrieval test

The latest stored record contains eight attempted conditions: six completed executions, including three offline runs, and two quota failures. The Hillcrest agent condition was not attempted after the quota stop. These selected outcomes are not an estimate of long-term provider availability.

All nine PDFs fit into one chunk each. Since initial retrieval requests six chunks and the largest dossier contains four, every live run starts with the entire supplied dossier. Consequently, this experiment cannot show an advantage from adaptive retrieval under a constrained evidence window. In addition, five of the eight retrieval queries belong to cases with at most three documents, making $R@3$ permissive. This is why I also inspect the first-ranked document and treat retrieval performance as a basic functionality check.

9 Results and resource trade-offs

9.1 Retrieval and report agreement

The expected source is ranked first for all eight saved queries: $R@1 = 8/8$ and $R@3 = 8/8$. The queries use explicit terms such as "incident report" and "invoice total", so these results establish correct routing on the constructed examples rather than semantic generalization.

Dossier	Mode	Status	Matched / extra / missed	Quotes
Harbor	RAG	Completed	0 / 0 / 0	3/3
Harbor	Agents	Completed	0 / 0 / 0	3/3
Riverside	RAG	Completed	3 / 0 / 0	4/4
Riverside	Agents	HTTP 429	Not measured	–
Hillcrest	RAG	HTTP 429	Not measured	–
Hillcrest	Agents	Not attempted	Not measured	–

Table 4: Current live-run outcomes. "Quotes" counts findings whose quotation checks pass, including supported confirmations. A zero issue count on Harbor is expected.

The offline rules reproduce the expected category sets for all three dossiers: no issue in Harbor, three categories in Riverside and missing information in Hillcrest. This is a consistency check on explicitly constructed fields, not evidence of general document understanding. The successful Riverside RAG report contains three issue categories and one supported finding confirming that the required estimate is present.

9.2 Paired comparison on Harbor

Both live modes confirm the consistent date, the presence of the repair estimate and the correct total. The agent run selects the calculator to add EUR 450 and EUR 600, drafts its report, and obtains a verifier assessment. It uses three model calls, compared with one for RAG.

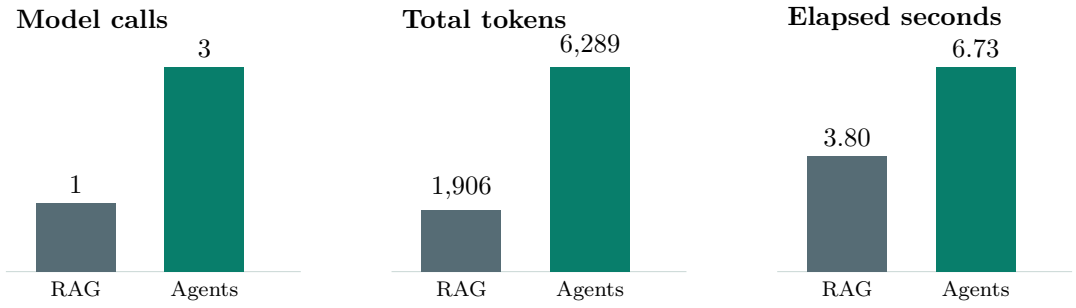


Figure 5: Resource use for the single paired Harbor comparison. Panels use independent linear scales. There is one recorded run per method, so no variability estimate is available.

The agent-to-baseline token ratio is $6289/1906 \approx 3.30$; elapsed time increases by $6.73/3.80 \approx 1.77$. Input tokens rise from 1,288 to 5,094 and output tokens from 618 to 1,195. Repeated evidence and verification context account for much of the added input. On this dossier, the benefit is an explicit calculation and audit step, not additional category coverage.

10 Case study: the Riverside dossier

10.1 Evidence relationships

The Riverside case illustrates why issue detection and interpretation should be separated. The declaration and incident report give different incident dates. The estimate contains a final total that exceeds the line-item sum. The copy document repeats the same invoice number but explicitly says that it is a resubmitted attachment.

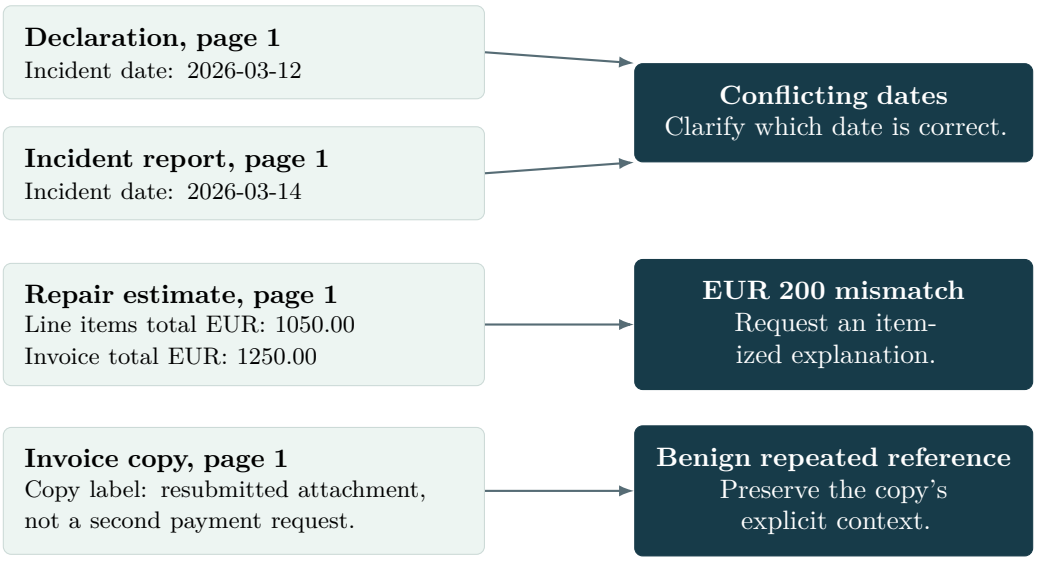


Figure 6: Source extracts and their interpretation in Riverside. The copy text is line-wrapped for presentation. These are excerpts from the synthetic documents, not independent annotations supplied to the model.

10.2 Recorded investigator behavior

The successful historical agent run, recorded at 19:04:59 UTC on 9 October, chose a subtraction action with $a = 1050.00$ and $b = 1250.00$. The calculator returned -200.00. Its report correctly described the final amount as EUR 200 higher than the line-item total. This illustrates that a deterministic tool produces a numerical result, while the model must interpret its sign and documentary context.

The run produced four findings: date contradiction, amount mismatch, required evidence present, and invoice copy as a resubmission. All four quotation checks passed, and the verifier supported all four findings. The run used three calls, 7,560 tokens and 9.18 seconds. No semantic revision was needed in this execution.

10.3 Historical evidence and the current record

This successful trace establishes that the investigator–verifier path ran on the contradictory dossier. It is not a replacement for the later current attempt, which failed with HTTP 429. The earlier run also preceded some response-format refinements. I therefore use it as a qualitative execution example, not as an interchangeable replicate in the primary cost comparison.

The distinction is important: retaining a useful historical success is compatible with reporting a later operational failure. Presenting only the former would conceal the incomplete coverage of the current experiment.

11 Software validation and reproducibility

11.1 Automated checks

The recorded core suite contains 22 passing tests. Two additional launcher tests were run separately and also passed. They verify that first-time setup creates an environment and blank configuration, while an existing configuration is preserved. Table 5 summarizes the types of checks rather than treating their number as a measure of model accuracy.

Area	Checked behavior
Document ingestion	Page extraction; invalid extensions, corrupt PDFs, binary text, size limits and empty input.
Evidence grounding	Valid and fabricated quotations; unknown document references; case-scoped retrieval.
Tool execution	Exact decimal operations; date difference; rejected unsupported operations.
Graph orchestration	Model-driven tool path with a controlled provider; revision and second review; unsupported action rejection; cancellation.
Provider adapter	JSON parsing; serving-schema preservation; token accounting; HTTP 429 without retries.
API lifecycle	Case upload, consent requirement, offline run, human review and JSON/Markdown/PDF export.
Local boundaries	Configuration without secrets; rejected unexpected host and cross-origin writes.
First-time launch	Environment creation and preservation of an existing configuration.

Table 5: Software validation coverage. Automated model-dependent tests use controlled responses; live model behavior is evaluated separately.

An HTTP smoke test also exercised a real Uvicorn listener and an HTTP client in the same execution context. The homepage, JavaScript, stylesheet, configuration, case list, evaluation and OpenAPI schema routes returned HTTP 200. This verifies serving and routing, not visual browser behavior. A complete cross-browser rendering check is not part of the saved validation record.

11.2 Reproducible artifacts

The repository includes source code, synthetic PDFs and their generation script, expected labels, saved experiment traces, test output, dependency ranges and an exact environment snapshot. Synthetic PDFs are generated with ReportLab's invariant output setting, and source hashes are preserved in runs. The report's figures are vector diagrams and plots embedded directly in the LaTeX source.

Temperature zero reduces one source of sampling variation, but does not guarantee identical outputs from a remotely served model. Latency also depends on provider load and network conditions. Reproduction therefore means rerunning the protocol with recorded configuration and comparing outcomes, not assuming byte-identical responses.

11.3 Demonstration and configuration

The local application runs through `start.cmd` or `python run.py`. A self-contained HTML demonstration replays recorded results without calling a provider. It is labelled as recorded, includes failed attempts, and distinguishes an available historical success from the latest execution. The offline template mode is a third, separate behavior.

Provider credentials are held in a local `.env` file and excluded from the source archive. The file is loaded at process startup, and an existing environment variable takes precedence. Changes therefore require a server restart. The later model-access diagnosis confirmed that a valid key can coexist with an unavailable model; configuration and quota failures must be distinguished from analysis failures.

12 Discussion and future work

12.1 What the results establish

The platform connects source files to structured findings, model-selected tools, verifier assessments and human review. Live runs demonstrate calculator selection and separate verification. Controlled tests exercise the bounded revision path, which the successful live examples did not require.

The paired Harbor result shows the cost of that additional structure: three calls and about 3.30 times the tokens for the same issue-category outcome. The evidence supports inspectability and executable orchestration. It does not establish better factual accuracy, more complete retrieval or fewer unsupported claims than the baseline.

12.2 Threats to validity

Small, development-used corpus. Three controlled dossiers do not represent the diversity of real documents. Their labels and formatting influenced development, and no independent held-out sample was used.

Full initial evidence. Every dossier fits inside the initial six-chunk window. The experiment therefore tests reasoning over available evidence. Longer dossiers are needed to measure gains from adaptive retrieval.

Coarse measures. Category agreement can conceal explanation errors or penalize reasonable alternative labels. Quote checks can pass for irrelevant evidence. Independently annotated claims and passages would provide a stronger assessment.

Correlated verification. The roles share a model and context, so agreement may reflect a shared error. Expert review or an independent checker is needed to assess the verifier's reliability.

Incomplete execution coverage. Quota failures prevented a complete current comparison, and each completed condition has only one retained current run. The observations do not support confidence intervals, robustness estimates or a general latency claim.

12.3 Prioritized extensions

I would first build a held-out collection of longer dossiers with independently annotated claims, ambiguous dates, legitimate charges and paraphrased requirements. Fixed retrieval, adaptive retrieval and investigator–verifier orchestration should then be compared under equal token or call budgets.

A second experiment would measure the verifier directly: inject known incorrect interpretations into otherwise valid, well-cited drafts, then record acceptance, rejection and correction rates. This isolates semantic verification from simple quotation matching.

Engineering priorities include semantic-retrieval ablation, persistent indexing, OCR, table extraction, adversarial-document handling, authentication and durable jobs. Measured failure modes should determine their order.

13 Conclusion

CaseTrace combines source-grounded investigation, bounded tools, structured reports, citation checks and human review in an executable platform. Recorded experiments quantify its resource use and define the limits of the available evidence.

The investigator and verifier make operations and objections explicit, while increasing calls, context and latency. They do not improve category coverage on the paired example. Larger, independently annotated dossiers are needed to establish when verification and adaptive retrieval produce measurable gains.

A Execution details and API reference

A.1 Principal parameters

Parameter	Implemented value used in the benchmark
Chunk length / stride	1,100 / 850 characters
Lexical vector dimension	2,048 hash buckets
Index / similarity	FAISS IndexFlatIP / normalized inner product
Initial retrieval / tool search	Up to 6 / up to 4 chunks
Retained evidence / page response	Last 20 entries / first 12,000 characters
Investigator actions / semantic revisions	At most 3 / at most 1
Default model-call budget	8 calls per run
Maximum requested output	3,500 tokens per model request
Benchmark model / temperature	Gemini 2.5 Flash / 0
Maximum report size	12 findings, 6 citations per finding

Table 6: Operational parameters. Serving limits are bounds, not a guarantee that all evidence will fit into a model's effective context.

A.2 Selected endpoints

Method	Path	Purpose
GET	`/api/config`	Non-secret configuration
POST	`/api/cases`	Upload a dossier
POST	`/api/demo`	Load synthetic cases
POST	`/api/cases/{case_id}/runs`	Start investigation
GET	`/api/runs/{run_id}`	Poll state and events
POST	`/api/runs/{run_id}/cancel`	Request cancellation
PUT	`/api/runs/{run_id}/review`	Save human assessment
GET	`/api/runs/{run_id}/export/{format}`	Export report
GET	`/api/evaluation`	Read saved evaluation

Table 7: Representative API operations. The OpenAPI schema describes the full interface.

A.3 Reproduction commands

From the project root, the following commands run the local application, software tests and offline evaluation on Windows:

```
.venv\Scripts\python.exe run.py
.venv\Scripts\python.exe scripts/test.py
.venv\Scripts\python.exe scripts/evaluate.py
```

An explicit live evaluation is requested with `scripts/evaluate.py -live`. It consumes provider quota and records failures when access is unavailable. The chosen model must be recorded with each new experiment; results obtained with a different model should not be silently pooled with the benchmark in this report.

B Evidence manifest

The following files provide the supporting implementation and experiment record. Paths are relative to the project root.

Artifact	Content
`casetrace/workflow.py`	Investigator, verifier, revision and stopping conditions.
`casetrace/retrieval.py`	Chunking, lexical vectors and optional semantic path.
`casetrace/documents.py`	Extraction, hashes and quotation validation.
`casetrace/provider.py`	Provider protocol, schemas and usage accounting.
`casetrace/app.py`	API, worker, consent and exports.
`scripts/generate_demo.py`	Synthetic PDF generation and intended labels.
`scripts/evaluate.py`	Comparison protocol and metric computation.
`artifacts/evaluation.json`	Current attempts, rankings and retained development history.
`artifacts/tests.xml`	Recorded 22-test core-suite outcome.
`artifacts/http-smoke.json`	Seven successful HTTP route checks.
`tests/test_bootstrap.py`	Two separately executed launcher checks.
`requirements-lock.txt`	Exact installed dependency snapshot.

Table 8: Implementation and experimental evidence used to prepare the report.

B.1 How to interpret the saved record

The `runs` collection stores the latest retained attempt for each executed case/mode pair. The `development_history` collection retains earlier attempts saved after history recording was introduced; it is not a complete log of every exploratory call. The report's primary outcome table uses `runs`. The successful historical Riverside trace is identified separately by its timestamp and is used only for the qualitative case study.

The numerical figures in this report are derived from these records. Token totals are provider-reported counts, not monetary costs. Durations are individual elapsed observations, not averages. A failed run without a final result has no report-quality score; it is not assigned a zero-quality or zero-token score.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, 2020. https://arxiv.org/abs/2005.11401.

[2] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.

[3] M. Douze et al., "The Faiss Library," arXiv:2401.08281, 2024. https://arxiv.org/abs/2401.08281.

[4] LangChain, "LangGraph overview," official documentation. https://docs.langchain.com/oss/python/langgraph/overview.

[5] Google, "OpenAI compatibility," Gemini API documentation. https://ai.google.dev/gemini-api/docs/openai.

[6] FastAPI, official documentation. https://fastapi.tiangolo.com/.